

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Last Name: CAÑETE
First Name: PHOEBIEN

Date: SEPTEMBER 28, 2026
Subject/Section: BCA182 B187

Problem and Requirements

The project simulates a room monitor on an STM32 Blue Pill using PlatformIO, STM32Cube HAL, and FreeRTOS. It reads temperature and humidity from a DHT22, relative light level from an ADC-connected photoresistor, and motion from a PIR. A rotary encoder selects one of four OLED pages. A buzzer alerts when temperature is below 18 °C or above 30 °C. After 15 seconds without PIR motion, the system enters INACTIVE; motion restores ACTIVE.

Requirement	Implementation	Verification
FR-01	SensorTask	FT-01
FR-02	DHT22	FT-02
FR-03	SensorTask; ADC1 photoresistor	FT-03
FR-04	MotionTask; PIR	FT-08
FR-05	DisplayTask; SSD1306	FT-01–FT-03
FR-06	InputTask; navigation queue	Navigation unit tests; FT-04 / FT-05
FR-07	AlarmTask; alarm logic; buzzer	Alarm unit tests; FT-06 / FT-07
FR-08	MotionTask; ACTIVE state bit	FT-08; state unit tests
FR-09	MotionTask; 15 s inactivity check	FT-09; state timeout tests
FR-10	MotionTask; PIR wake path	FT-10; state reactivation test

Wokwi substitutes simulated peripherals for physical hardware. The simulated LDR is adjusted in lux in Wokwi, but the firmware reports its ADC reading as a percentage of the 12-bit range, because no photometric calibration is performed. In Wokwi I could adjust the sensor stimuli and observe the OLED, buzzer, and serial output.

System Architecture and Design

DHT22 data is connected to PB0; the photoresistor analog output to PA0; PIR output to PB1; encoder CLK and DT to PB12 and PB13; SSD1306 I2C to PB6 and PB7; buzzer PWM to

PA2; and USART1 diagnostics to PA9 and PA10. Hardware setup resides in hardware.cpp; task entry points and independent decision logic are separated into modules.

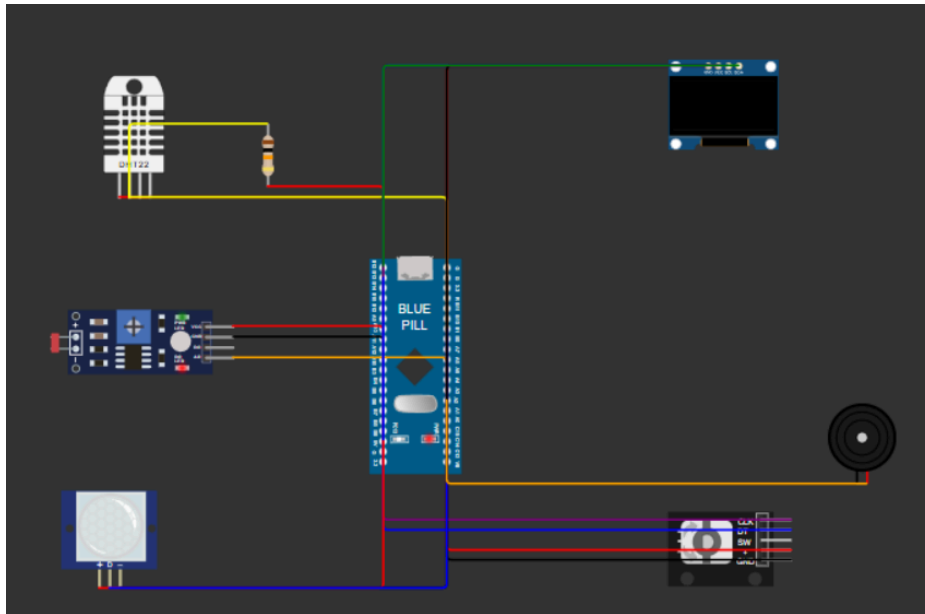


Figure 1. Wokwi circuit and pin connections

MotionTask owns the activity state. It updates EVENT_ACTIVE and EVENT_MOTION. ACTIVE enables OLED, encoder, and alarm behavior. After the inactivity threshold MotionTask clears EVENT_ACTIVE; DisplayTask blanks or powers down the OLED, while input and alarm handling pause. MotionTask continues to poll PIR and sets EVENT_ACTIVE when motion returns.

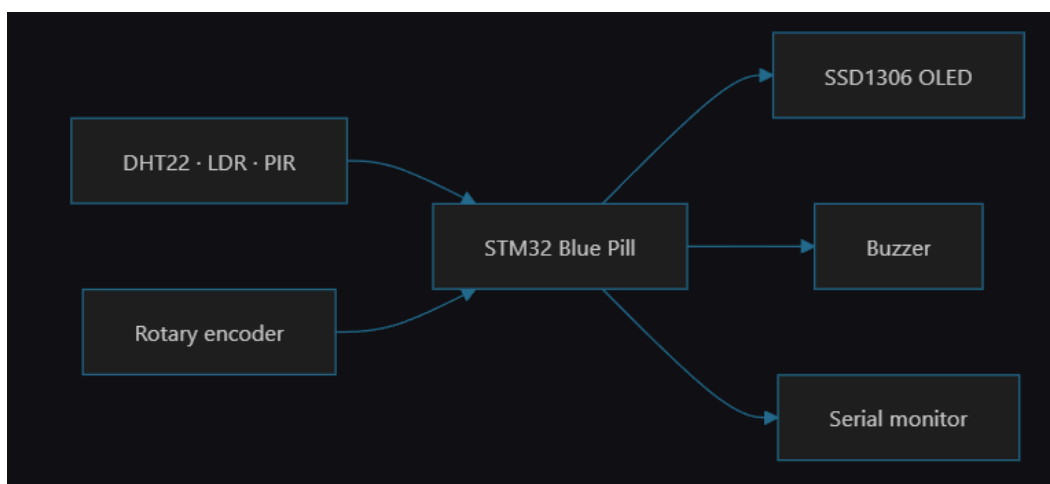


Figure 2. Sensors and user input feed the STM32, which controls the display, alarm, and diagnostics.

State machine. The system has two states, ACTIVE and INACTIVE, owned by MotionTask. It starts in ACTIVE. If no PIR motion is detected for 15 seconds, MotionTask clears EVENT_ACTIVE and the system enters INACTIVE. Any PIR motion while INACTIVE sets EVENT_ACTIVE and returns the system to ACTIVE. In ACTIVE, the OLED, encoder, and alarm operate normally. In INACTIVE, the OLED is blanked, and input and alarm handling pause, but PIR polling continues so that motion can wake the system.

FreeRTOS Architecture

The application creates five tasks with explicit priorities. A delay moves a task into Blocked; on expiry it becomes Ready and runs when selected. InputTask, DisplayTask, and AlarmTask can additionally wait for the ACTIVE event. The following table describes their actual roles.

Task	Trigger	Priority	Communication	Typical blocked condition
SensorTask	About 2 s	2	Two SensorData queues	vTaskDelayUntil
DisplayTask	50 ms check / updates	1	Sensor and mode queues; event bits	Delay or ACTIVE wait
InputTask	50 ms check / encoder	3	Mode queue; event bits	Delay or ACTIVE wait
MotionTask	100 ms PIR poll	3	ACTIVE and MOTION bits	vTaskDelayUntil
AlarmTask	Sensor arrival / state	2	Sensor queue; ALARM bit	Queue receive or ACTIVE wait

Motion and encoder inputs have the most immediate responsiveness needs. Sensor acquisition and alarm processing tolerate the roughly two-second sampling period. OLED refresh can occur at lower priority. The custom Wokwi scheduler port advances a 20 Hz tick with TIM3 but does not perform an interrupt-mode context switch; a runnable task that fails to block can delay a newly ready task. This limits claims of strict preemptive timing.

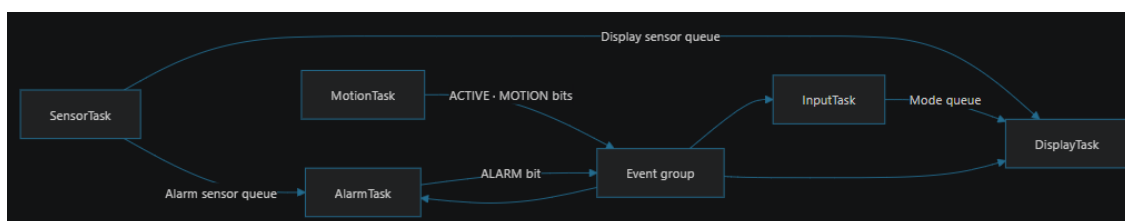


Figure 3. FreeRTOS Architecture Diagram

SensorTask overwrites two separate one-item queues so DisplayTask and AlarmTask each receive the newest complete SensorData structure. InputTask overwrites the display-mode queue. MotionTask maintains ACTIVE and MOTION bits; AlarmTask maintains ALARM. A

FreeRTOS mutex protects USART1 writes from interleaving when multiple tasks print diagnostics.

Task states in this system. SensorTask is *Blocked* in `vTaskDelayUntil()` for most of each 2 s period. When the wake time arrives it becomes *Ready*, and it is *Running* while it reads the DHT22 and ADC. DisplayTask, InputTask, and AlarmTask are *Blocked* on the `EVENT_ACTIVE` bit while the system is `INACTIVE`. A task is *Ready* when its delay has expired but a higher-priority task, such as MotionTask, is still running, so it waits for the CPU. The difference is that a Blocked task is waiting for an event or timeout and cannot run, while a Ready task can run and is only waiting to be selected.

`vTaskDelayUntil()` vs. `vTaskDelay()`. `vTaskDelay(2000)` waits 2000 ms *after* the work finishes, so each cycle takes 2000 ms plus the execution time of the DHT22 read and ADC sampling. The sampling instants therefore drift later over time. `vTaskDelayUntil()` wakes the task at absolute times measured from a stored reference tick (`lastWakeTime`), so the period stays 2 s regardless of how long the work takes, as long as the work is shorter than the period. SensorTask uses it for this reason. MotionTask uses it for a steady 100 ms PIR poll.

Queues vs. global variables. SensorData contains four fields written together by SensorTask. With plain globals, DisplayTask could read the struct halfway through an update and see a new temperature with an old humidity. A queue copies the whole structure atomically. I used overwrite queues of length one because only the newest sample matters, and I used two separate queues so DisplayTask and AlarmTask each receive every sample.

Mutex. The shared resource is the USART1 transmit path used for diagnostic output. Without the mutex, one task can be interrupted partway through a line while another task writes, so characters from the two messages interleave into a garbled line. The mutex makes each message an uninterrupted unit.

Implementation

DHT22 pulse timing uses a dedicated 1 MHz TIM4 counter. SensorTask enters a critical section for the timing-sensitive read and samples the ADC afterward. If either acquisition fails, it does not publish a zero-valued measurement because SensorData has no validity flags. The display treats a measurement as stale after five seconds.

The OLED belongs to DisplayTask and the buzzer belongs to AlarmTask, avoiding competing hardware writes. An encoder interrupt records turns without calling FreeRTOS from its high-priority ISR; InputTask applies navigation. Temperature alarm and system-state

decisions are separated from hardware access and exercised by native tests. `vTaskDelayUntil` uses a scheduled wake time for periodic sampling, avoiding the accumulated drift caused by delaying a fixed amount after each iteration.

Implementation: source modules in `src/` and interfaces in `include/`.

Unit tests: `test/test_alarm/`, `test/test_navigation/`, and `test/test_state/`.

Circuit and simulator setup: `diagram.json` and `wokwi.toml`.

Functional observations: `docs/functional-verification.md`.

Public overview: [README.md](#).

Vendor materials:

STM32CubeF1 framework (PlatformIO package 1.8.7)

FreeRTOS kernel sources (version 10.3.1)

PlatformIO documentation

Wokwi documentation

Verification and Testing

From the repository root, run `pio run -e bluepill_f103c8` for the firmware, `pio test -e native` for host-side decision tests, and `pio check -e bluepill_f103c8` for Cppcheck. Build the binary before starting Wokwi. Repeat the ten stimuli in `docs/functional-verification.md` and compare actual OLED, buzzer, and serial output.

Automated unit tests

The student reported 13/13 passing with `pio test -e native`: five alarm-limit cases (including both exact boundaries), four clockwise/counterclockwise navigation cases (including wraparound), and four activity/timeout cases. These tests exercise deterministic logic, not simulated electrical behavior.

Wokwi functional tests

The student reported all ten FT-01 to FT-10 checks passing. The recorded stimuli and observations are stored separately in `docs/functional-verification.md`. The entries cover DHT22 temperature and humidity updates, a changed light percentage, both navigation directions, alarm activation and clearance, PIR motion, timeout, and wake.

Test ID	Input / Stimulus	Expected Result	Actual Observation	Result
FT-01	Set DHT22 temperature to 28 °C	OLED displays 28 °C	OLED displayed 28 °C	PASS
FT-02	Set DHT22 humidity to 70%	OLED displays 70% humidity	OLED displayed 70% humidity	PASS
FT-03	Change LDR setting from 500 lux to 100 lux in Wokwi	Displayed light level changes	OLED light-level percentage changed	PASS
FT-04	From Temperature page, rotate encoder clockwise once	Humidity page appears	OLED switched to the Humidity page	PASS
FT-05	From Humidity page, rotate encoder counterclockwise once	Temperature page appears	OLED switched to the Temperature page	PASS
FT-06	Set DHT22 temperature to 31 °C	Alarm activates and buzzer sounds	Alarm activated and buzzer sounded	PASS
FT-07	Return DHT22 temperature to 25 °C	Alarm clears and buzzer stops	Alarm cleared and buzzer stopped	PASS
FT-08	Trigger PIR motion	System is ACTIVE	System was ACTIVE (OLED on)	PASS
FT-09	Wait 15 s after PIR output returns LOW	System enters INACTIVE	System entered INACTIVE (OLED blank)	PASS
FT-10	Trigger PIR motion while INACTIVE	System returns to ACTIVE	System returned to ACTIVE (OLED on)	PASS

Deliberate FreeRTOS Fault Experiments

Experiment	Expected effect	Actual observation
Remove blocking	A task that runs continuously can occupy the CPU and delay other tasks. This is especially visible with this project's custom scheduler, which relies on tasks blocking or yielding.	After removing the delay from SensorTask, the simulation became less responsive and other task outputs were delayed. Restoring the delay returned the system to normal operation.
Raise priority	A frequently running task may run ahead of lower-priority tasks, increasing their response time. If it still blocks regularly, the effect may be small.	After assigning SensorTask an unnecessarily high priority, OLED updates and encoder input became slower. Restoring its original priority improved responsiveness.
Remove mutex	Concurrent serial messages <i>may</i> become mixed. A short run can also appear normal because timing may not expose a collision.	After removing serial mutex protection, some diagnostic messages appeared mixed in the Serial Monitor. Restoring the mutex kept complete messages together.

Static Code Analysis

The student reported that `pio check -e bluepill_f103c8` passed with 0 high, 0 medium, and 41 low style findings after cleanup. The firmware build also succeeded; reported usage was 12,104 of 20,480 bytes RAM and 22,076 of 65,536 bytes flash. These results should be rerun after any subsequent code changes.

Finding	File	Cause	Action
unusedFunction / low	src/alarm.cpp:12; src/cubemx/stm32f1xx_it.c:85	Task entries and vector handlers are invoked outside the analyzer's visible local call paths.	Retain required FreeRTOS tasks, HAL hooks, and handlers.
constParameterPointer / low	src/cubemx/stm32f1xx_hal_msp.c:91; src/hardware.cpp:197	Analyzer sees read-only use of HAL handle parameters.	Retain callback signatures expected by HAL.
cstyleCast / low	src/serial_log.cpp:63	C-style cast appears in FreeRTOS semaphore macro expansion.	Do not rewrite vendor macro.
useStlAlgorithm / low	src/ssd1306.cpp:88	Explicit ordered command loop could be expressed through STL.	Retain loop for early exit on I2C error.
constVariable, unused helper / low	src/rtos_hooks.cpp:27; earlier src/hardware.cpp:24	Immutable local buffer and obsolete uncalled helper.	Made array const; removed helper and declaration.

Engineering Discussion

Separate one-item queues prioritize the latest sample over a historical backlog and avoid one consumer taking another's data. The four-field `SensorData` cannot represent partial validity, so a failed DHT22 or ADC read suppresses both outputs; per-sensor validity would improve this. ADC percentage is deliberately distinguished from lux because no photometric calibration is performed. Adding a `StateTask` would add one more task and one more queue for no additional behavior. Keeping the transition in `MotionTask`, while the pure decision function (`evaluateSystemState`) is separately unit-tested, keeps the design smaller. The trade-off is that `MotionTask` carries two responsibilities, PIR polling and state management.

The custom scheduler port was chosen for simulator behavior but differs from the standard Cortex-M port. Its timing and robustness should be validated before using physical hardware. The DHT22 critical section can postpone lower-priority interrupts briefly. The serial mutex prevents concurrent task writes from mixing messages, though it can serialize tasks during lengthy output.

Development encountered native test setup issues on Windows: `PlatformIO` requires a singular `test/` folder and a GCC/G++ toolchain on `PATH`. The project was also cleaned to remove duplicate task implementations from `main.cpp` and obsolete compatibility files. After cleanup the student-reported build and tests passed; simulator behavior should be rechecked against the latest firmware binary.

Conclusion

This project decomposed a simulated STM32 room monitor into five cooperating FreeRTOS tasks, using queues for measured data, an event group for shared state, and a mutex for serial output. Thirteen native unit tests and 13 Wokwi functional tests pass, and three deliberate fault experiments showed what happens when blocking, priority, or mutex protection is misused. I learned that priorities express scheduling urgency, that a task must block to let others run, especially on a non-preemptive port, and that separating decision logic from hardware makes it testable. To improve the design I would add per-sensor validity flags to SensorData, move to the standard Cortex-M port, and validate the system on real hardware.